

Embodied Artificial Intelligence

HW2: Robotics Simulation Workflow

Department: Department of Automation

Class: Class 31

Name: Zuo Gou

ID: 2023012274

1 Part 1: Hello World

1.1 Reproduction

To reproduce the scene and generate the video, run the following command:

```
uv run part1/hello_world.py
```

1.2 Scene Construction

In this part, I constructed a basic Genesis scene using the Genesis framework. The scene consists of a table, four different robots, two object models, and a moving camera.

Table Construction: The table is built using five boxes: one tabletop and four legs. The tabletop is placed at the center $(0, 0, 0.825)$ with size $(1, 1, 0.05)$, and each leg is positioned at the corners, calculated based on the tabletop and leg sizes. All table components are fixed in the scene.

Robots:

- **Go2 (Quadruped):** Loaded from its URDF, placed at $(0, -1, 0.42)$, facing the table center (rotated 90° about the z -axis). The default joint positions are saved for resetting.
- **G1 (Humanoid):** Loaded from its URDF, placed at $(-0.7, 0, 0.85)$, facing the table center (no rotation needed). Default joint positions are saved.
- **Aloha (Mobile Manipulator):** Loaded from its URDF, placed at $(0, 1, 0.1)$, facing the table center (rotated -90° about the z -axis). Default joint positions are saved.
- **Franka (Robotic Arm):** Loaded from its URDF, placed at $(-0.2, 0, 0.85)$ on the table edge, facing the table center. This arm is fixed in the scene. Default joint positions are saved.

Objects:

- **Airplane:** A mesh model loaded from `assets/models/Airplane/airplane.obj`, scaled to 0.003, placed above the table at $(0.2, -0.2, 0.9)$, facing the $+x$ direction (rotated 90° about x and z).
- **Duck:** A mesh model loaded from `assets/models/duck/duck.obj`, scaled to 0.0003, placed at $(0.2, 0.2, 0.9)$, also facing the $+x$ direction.

Camera: A camera is added with resolution 960×720 , field of view 60° , near plane 0.1, and far plane 10.0. During simulation, the camera orbits the scene in a circular path of radius 2 at height 2.8, always looking at the table center $(0, 0, 0.5)$. The camera records a video as it moves.

Robot Control: If the robots are fixed, their joint positions are reset to the default pose at each step using `control_dofs_position`. This keeps them static for visualization.

1.3 Results & Analysis

The constructed scene visually demonstrates a typical tabletop manipulation setup in simulation. The table is at the center, with four robots (Go2, G1, Aloha, Franka) placed around it, each facing the center for potential interaction. The airplane and duck models are placed on the tabletop as objects to manipulate or observe. The camera orbits the scene, providing a dynamic and comprehensive view of all elements.

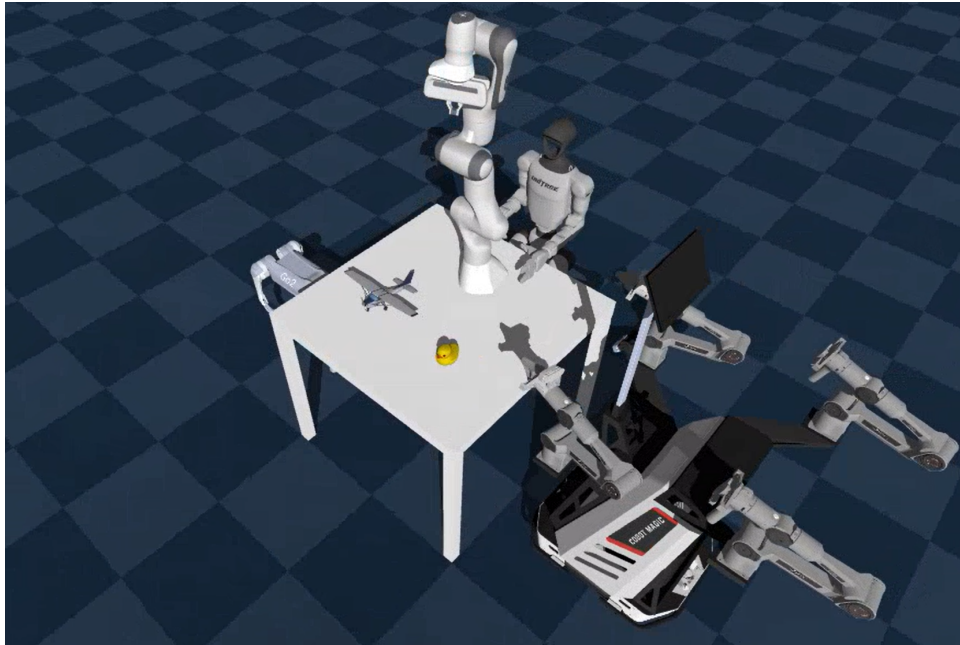


fig1: Hello World Scene: Table, Robots, and Objects

The output video, `part1/hello_world.mp4`, shows the camera smoothly circling the scene, highlighting the spatial arrangement and orientation of all robots and objects. This setup provides a foundation for more complex simulation tasks in later assignments.

2 Part 2: Sensors

2.1 IMU Sensor (Part 2.1)

2.1.1 Reproduction

To run the IMU simulation and generate the acceleration plot:

```
uv run part2/imu.py
```

2.1.2 Implementation

In this experiment, I attached a simulated IMU sensor to the end-effector link (panda_link7) of a Franka Emika Panda robot. The sensor was configured with realistic noise parameters, including acceleration axis skew, bias, and Gaussian noise, to mimic real-world sensor characteristics.

To generate dynamic acceleration data, I implemented a kinematic controller that moves the end-effector along a circular path of radius 0.15 m centered at (0.4, 0.0, 0.5). The robot's joint positions were computed using inverse kinematics at each time step. During the simulation, I recorded both the raw acceleration readings from the IMU and the ground-truth acceleration provided by the physics engine.

2.1.3 Analysis

The IMU sensor was attached to the Franka arm's end-effector. As shown in Figure 2, the raw IMU measurements (blue) exhibit significant noise compared to the ground truth acceleration (orange). This demonstrates the importance of filtering or sensor fusion in real-world robotics applications.

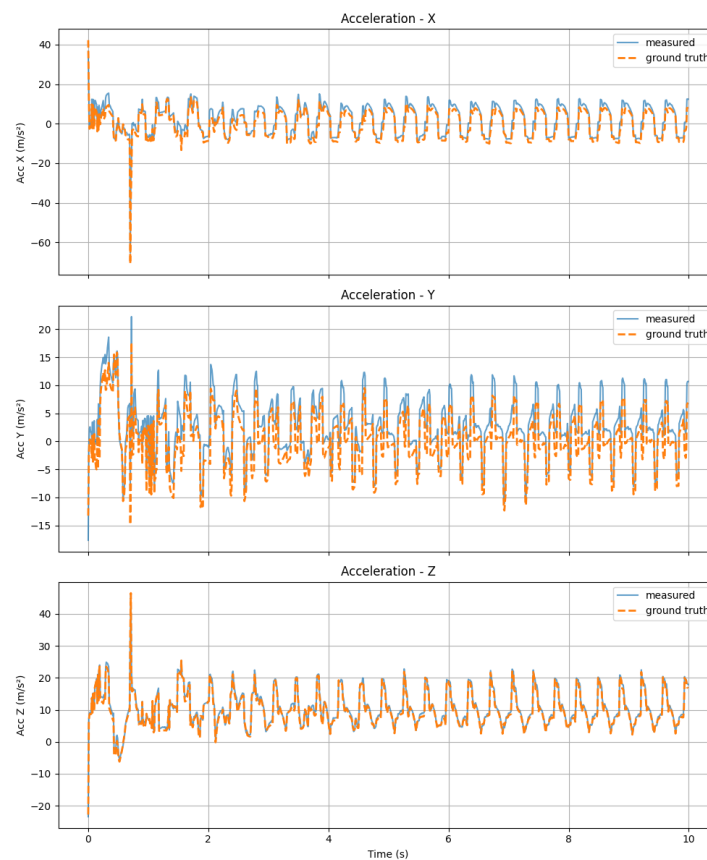


fig2: IMU Acceleration Plot: Noisy vs Ground Truth

2.2 LiDAR Sensor (Part 2.2)

2.2.1 Reproduction

To simulate the maze navigation and record sensor data:

```
uv run part2/lidar.py
```

To visualize the recorded depth and LiDAR data:

```
uv run scripts/depth_viz.py --data_path part2/depth_maze_data.npz
uv run scripts/lidar_viz.py --data_path part2/lidar_maze_data.npz
```

2.2.2 Implementation

I constructed a static maze environment using box primitives based on a 5×5 grid layout. A Breadth-First Search (BFS) algorithm was implemented to find the shortest path from the start position 'S' to the goal 'G'.

A Unitree Go2 robot was spawned at the start location. I attached two perception sensors to its base link:

- **LiDAR:** A spherical scanning LiDAR with a 360° horizontal FOV and 30° vertical FOV, configured to sample 512×16 points per scan.
- **Depth Camera:** A camera sensor outputting depth maps with a resolution of 160×120 and a horizontal FOV of 60° .

The robot autonomously navigates the calculated path using a simple waypoint tracking controller, which adjusts the robot's yaw rate to face the next waypoint while maintaining a constant forward velocity. Sensor data is recorded at every simulation step for post-processing.

2.2.3 Visualization & Analysis

The robot navigates through the maze while capturing environmental data.

- **Depth Camera:** Captures the distance to obstacles pixel-by-pixel. The visualization shows clear outlines of the maze walls.
- **LiDAR:** Provides a 360-degree point cloud. The visualization demonstrates how the robot perceives the geometry of the maze in real-time.

The recorded sensor data is saved in `part2/depth_maze_data.npz` and `part2/lidar_maze_data.npz`. The visualization videos are `part2/depth_viz.mp4` and `part2/lidar_viz.mp4`.

3 Part 3: Tasks

3.1 Go2 Locomotion Training

3.1.1 Reproduction

To train the policy:

```
uv run part3/go2_train.py --max_iterations 1001
```

3.1.2 Environment & Training

The training environment is defined in `part3/go2_env.py`, inheriting from the Genesis simulation framework. The robot is a Unitree Go2 quadruped with 12 degrees of freedom.

Observation Space: The observation vector (dim=45) includes:

- Base linear velocity (3)
- Base angular velocity (3)
- Projected gravity vector (3)
- Joint positions (12) and velocities (12)
- Previous actions (12)

Action Space: The policy outputs 12 joint position targets, which are processed through a PD controller ($K_p = 20.0$, $K_d = 0.5$) with an action scale of 0.25.

Reward Design: The reward function encourages tracking the user-provided velocity commands while maintaining stability and energy efficiency. Key components include:

- **Tracking Rewards:** `tracking_lin_vel` (1.0) and `tracking_ang_vel` (0.2) penalize deviations from the target command $c = (v_x^c, v_y^c, \omega^c)$.
- **Stability Penalties:** `lin_vel_z` (-1.0) penalizes vertical body movement, and `base_height` (-50.0) keeps the base near 0.3m.
- **Regularization:** `action_rate` (-0.005) and `similar_to_default` (-0.1) encourage smooth motions and natural posture.

Training: The policy was trained using PPO (Proximal Policy Optimization) with an Actor-Critic network (hidden layers: [512, 256, 128]) for 1001 iterations.

3.2 Evaluation

3.2.1 Reproduction

To evaluate the trained policy and generate plots/videos:

```
uv run part3/go2_eval.py --ckpt 1000
```

3.2.2 Command Tracking Analysis

I evaluated the trained policy with three commands:

- $c_1 = (1, 0, 0)$
- $c_2 = (0, 0.5, 0)$
- $c_3 = (0, 0, 1)$

As shown in Figure 3, the robot successfully tracks the commanded velocities.

- For c_1 , the forward velocity v_x converges to 1.0 m/s.
- For c_2 , the lateral velocity v_y tracks 0.5 m/s.
- For c_3 , the yaw rate ω follows the command of 1.0 rad/s.

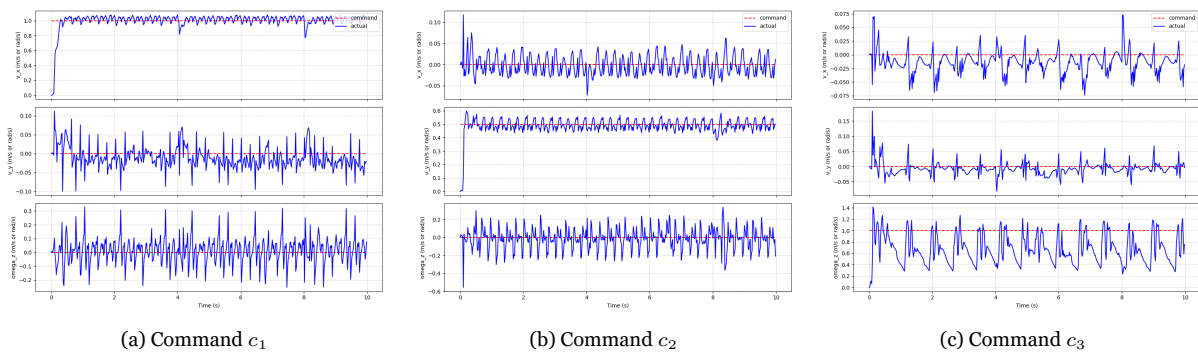


fig3: Commanded vs Actual Velocities

The top-down videos for each command are provided in the part3 folder.

3.3 Maze Navigation

3.3.1 Reproduction

To run the maze navigation demo:

```
uv run part3/go2_maze_run.py --ckpt 1000
```

3.3.2 Implementation

Maze Environment: The Go2MazeEnv class extends the standard training environment by procedurally generating a maze terrain. The maze layout is defined by a grid string, where walls are constructed using static box primitives. The start 'S' and goal 'G' positions are parsed from the grid to define the navigation task.

Navigation Logic: The navigation task is solved using a hierarchical approach:

1. **Global Planner:** A Breadth-First Search (BFS) algorithm computes the shortest path of grid cells from the start to the goal. The centers of these cells serve as waypoints.
2. **Local Controller:** A pure pursuit-style controller calculates the velocity command (v_x, v_y, ω) needed to reach the next waypoint.

$$v_{cmd} = K_{lin} \cdot R_{yaw}^T \cdot (p_{target} - p_{current}) \quad (1)$$

where R_{yaw} is the robot's heading rotation matrix. The computed commands are fed into the trained locomotion policy to drive the robot.

3.3.3 Results & Analysis

The robot successfully navigated from S to G using the trained locomotion policy. The navigation logic (e.g., waypoint tracking) effectively guided the robot through the corridors. The attached sensors (LiDAR and Depth Camera) provided continuous environmental perception during the run.

- Top-down video: part3/maze_topdown.mp4
- Depth visualization: part3/maze_depth_vis.mp4
- LiDAR visualization: part3/maze_lidar_vis.mp4